

A **Stored Procedure** is a precompiled collection of SQL statements stored in a database and executed as a single unit. It helps improve performance, reusability, security, and maintainability.

General Syntax of a Stored Procedure

In MySQL

```
DELIMITER //
```

```
CREATE PROCEDURE procedure_name(  
    IN param1 datatype,  
    OUT param2 datatype,  
    INOUT param3 datatype  
)  
BEGIN  
  
    -- SQL statements  
  
END //
```

```
DELIMITER ;
```

Explanation of Each Part

1. **DELIMITER //**

By default, MySQL uses ; to end SQL statements.

Since a stored procedure contains multiple SQL statements ending with ;, we temporarily change the delimiter to //.

```
DELIMITER //
```

After creating the procedure, we switch it back:

```
DELIMITER ;
```

2. **CREATE PROCEDURE**

Used to create a new stored procedure.

```
CREATE PROCEDURE procedure_name()
```

Example:

```
CREATE PROCEDURE GetEmployees()
```

3. Parameters

Stored procedures can accept parameters.

Types of Parameters

Type	Meaning
IN	Input value passed to procedure
OUT	Returns a value from procedure
INOUT	Works as both input and output

Example:

```
(IN emp_id INT)
```

4. BEGIN ... END

All SQL statements of the procedure are written inside this block.

```
BEGIN  
  
    SQL statements;  
  
END
```

Example 1 — Simple Stored Procedure

```
DELIMITER //  
  
CREATE PROCEDURE ShowStudents()  
BEGIN  
  
    SELECT * FROM Students;  
  
END //  
  
DELIMITER ;
```

Explanation

- `ShowStudents()` → Procedure name
- `SELECT * FROM Students;` → Fetches all records
- No parameters used

Execute Procedure

```
CALL ShowStudents();
```

Example 2 — Procedure with IN Parameter

```
DELIMITER //  
  
CREATE PROCEDURE GetStudentById(  
    IN sid INT  
)  
BEGIN  
  
    SELECT *  
    FROM Students  
    WHERE StudentID = sid;  
  
END //  
  
DELIMITER ;
```

Explanation

- sid is an input parameter
- User passes student ID while calling procedure

Execute

```
CALL GetStudentById(101);
```

Example 3 — Procedure with OUT Parameter

```
DELIMITER //  
  
CREATE PROCEDURE GetTotalStudents(  
    OUT total INT  
)  
BEGIN  
  
    SELECT COUNT(*) INTO total  
    FROM Students;  
  
END //  
  
DELIMITER ;
```

Execute

```
CALL GetTotalStudents(@t);  
  
SELECT @t;
```

Explanation

- OUT total INT returns value
- COUNT(*) result stored in total
- @t is a session variable

Example 4 — Procedure with INOUT Parameter

```
DELIMITER //

CREATE PROCEDURE IncreaseNumber(
    INOUT num INT
)
BEGIN
    SET num = num + 10;
END //

DELIMITER ;
```

Execute

```
SET @x = 5;

CALL IncreaseNumber(@x);

SELECT @x;
```

Output

```
15
```

Advantages of Stored Procedures

Advantage	Description
Faster Execution	Precompiled SQL improves performance
Reusability	Write once, use many times
Security	Users can execute procedure without direct table access
Easy Maintenance	Business logic stored centrally
Reduced Network Traffic	Multiple SQL statements executed together

Trigger

A **trigger in SQL** is a special type of stored procedure that **automatically executes (fires)** when a specific event occurs in a database table or view.

◆ What is a Trigger?

A trigger is used to enforce rules, maintain audit logs, or automatically update related data when changes happen in a table.

◆ Types of Triggers

1. **BEFORE Trigger**
 - Executes **before** an INSERT, UPDATE, or DELETE operation.
2. **AFTER Trigger**
 - Executes **after** the operation is completed.
3. **INSTEAD OF Trigger**
 - Executes **instead of** the original operation (commonly used with views).

In databases, a **trigger** is a piece of SQL code that runs **automatically** when something happens in a table.

Think of it like:

“If this event happens, do this automatically.”

Common events:

- INSERT → when new data is added
 - UPDATE → when data changes
 - DELETE → when data is removed
-

Simple Real-Life Example

Imagine a bank system.

When money is added to an account, you want to automatically save a record in a history table.

Instead of writing this every time manually, we create a **trigger**.

◆ Basic Syntax

```
CREATE TRIGGER trigger_name
BEFORE | AFTER | INSTEAD OF
INSERT | UPDATE | DELETE
ON table_name
FOR EACH ROW
BEGIN
    -- SQL statements
END;
```

◆ Example

```
CREATE TRIGGER before_insert_employee
BEFORE INSERT
ON employees
FOR EACH ROW
BEGIN
    SET NEW.created_at = NOW();
END;
```

Example Tables

Account Table

Id name balance

1 Rahul 5000

Transaction History Table

Msg

Rahul deposited money

Trigger Example

```
CREATE TRIGGER save_history
AFTER UPDATE ON account
FOR EACH ROW
BEGIN
```

```
INSERT INTO history(msg)
VALUES ('Money updated in account');
END;
```

What This Means

- `CREATE TRIGGER save_history`
→ creates a trigger named `save_history`
 - `AFTER UPDATE ON account`
→ run trigger after account table is updated
 - `FOR EACH ROW`
→ run for every changed row
 - `INSERT INTO history`
→ automatically insert data into history table
-

What Happens Automatically

If someone runs:

```
UPDATE account
SET balance = 7000
WHERE id = 1;
```

Then the trigger automatically runs and inserts:

msg

Money updated in account

into the `history` table.

Subquery

A **subquery** in SQL is a query written **inside another SQL query**.

It is also called:

- **Inner query**
- **Nested query**

The subquery executes first, and its result is used by the main query.

◆ Basic Syntax

```
SELECT column_name
FROM table_name
WHERE column_name operator (
    SELECT column_name
    FROM another_table
);
```

◆ Example 1 — Simple Subquery

Find employees who earn more than the average salary.

```
SELECT name, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

How it works

1. Inner query:

```
SELECT AVG(salary) FROM employees;
```

Suppose result = 50000

2. Main query becomes:

```
SELECT name, salary
FROM employees
WHERE salary > 50000;
```

Operators Used with Subqueries

Operator	Use
=	Single value
IN	Multiple values
ANY	Compare with any value
ALL	Compare with all values
EXISTS	Checks existence

◆ Difference Between Subquery and Join

Subquery	Join
Query inside query	Combines tables
Easier to read sometimes	Often faster
Can be nested	Better for relationships
May be slower	Usually optimized

◆ Real-Life Example

Find customers who placed orders

```
SELECT customer_name
FROM customers
WHERE customer_id IN (
    SELECT customer_id
    FROM orders
);
```

Index

index is a data structure that improves the speed of data retrieval operations on a table (similar to an index in a book). Instead of scanning the entire table, MySQL can quickly locate rows using the index.

🔑 Why use an index?

- Speeds up `SELECT` queries
- Helps with `WHERE`, `JOIN`, `ORDER BY`, and `GROUP BY` clauses
- Reduces disk I/O

⚠ Trade-off: Indexes use extra storage and can slow down `INSERT`, `UPDATE`, and `DELETE`.

📌 Types of indexes in MySQL

1. Primary Index (PRIMARY KEY)

- Automatically created when you define a primary key
- Unique and NOT NULL

```
CREATE TABLE users (  
  id INT PRIMARY KEY,  
  name VARCHAR(100)  
);
```

2. Unique Index

- Ensures all values in a column are unique

```
CREATE UNIQUE INDEX idx_email ON users(email);
```

3. Normal (Non-Unique) Index

- Improves search performance but allows duplicate values

```
CREATE INDEX idx_name ON users(name);
```

4. Composite Index (Multi-column)

- Index on multiple columns

```
CREATE INDEX idx_name_age ON users(name, age);
```

☞ Order matters! (name, age) is different from (age, name).

5. Full-Text Index

- Used for text searching

```
CREATE FULLTEXT INDEX idx_content ON articles(content);
```

⚙️ How to create and manage indexes

Create index

```
CREATE INDEX idx_column ON table_name(column);
```

Show indexes

```
SHOW INDEX FROM table_name;
```

Drop index

```
DROP INDEX idx_column ON table_name;
```

🚀 Example

```
SELECT * FROM users WHERE name = 'John';
```

Without an index → full table scan

With an index on name → fast lookup

Keys in SQL

1. Primary Key

A **Primary Key** uniquely identifies each row in a table.

Rules

- Must contain unique values
- Cannot contain `NULL`
- One primary key per table

Example

```
CREATE TABLE Students (  
    student_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    age INT  
);
```

Data

student_id	name	age
101	Amit	20
102	Neha	21

Here, `student_id` is the primary key because every student has a unique ID.

2. Foreign Key

A **Foreign Key** creates a relationship between two tables.

It refers to the primary key of another table.

Example

```
CREATE TABLE Courses (  
    course_id INT PRIMARY KEY,  
    course_name VARCHAR(50)  
);
```

```
CREATE TABLE Enrollments (  
    -- Foreign Key to Courses table  
    -- Primary Key for Enrollments table
```

```
    enroll_id INT PRIMARY KEY,  
    student_id INT,  
    course_id INT,  
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)  
);
```

Here:

- `course_id` in `Courses` is the primary key
- `course_id` in `Enrollments` is the foreign key

This ensures only valid courses can be used.

3. Candidate Key

A **Candidate Key** is a column that can uniquely identify rows.

A table can have multiple candidate keys.

Example

```
CREATE TABLE Employees (  
    emp_id INT,  
    email VARCHAR(100),  
    passport_no VARCHAR(20)  
);
```

If all three values are unique:

- `emp_id`
- `email`
- `passport_no`

then all are candidate keys.

One of them becomes the primary key.

4. Alternate Key

Candidate keys not chosen as the primary key are called **Alternate Keys**.

Example

If:

- `emp_id` is chosen as Primary Key

then:

- `email`
- `passport_no`

become alternate keys.

5. Composite Key

A **Composite Key** uses multiple columns together to uniquely identify a row.

Example

```
CREATE TABLE Attendance (  
    student_id INT,  
    subject_id INT,  
    attendance_date DATE,  
    PRIMARY KEY (student_id, subject_id)  
);
```

Here:

- Neither `student_id` nor `subject_id` alone is unique
 - Together they form a composite key
-

6. Unique Key

A **Unique Key** ensures all values in a column are unique.

Unlike primary key:

- It can allow one `NULL` value (depends on DBMS)

Example

```
CREATE TABLE Users (  
    user_id INT PRIMARY KEY,  
    email VARCHAR(100) UNIQUE  
);
```

No two users can have the same email.

7. Super Key

A **Super Key** is any combination of columns that uniquely identifies rows.

Example

If `student_id` uniquely identifies students, then these are super keys:

- `student_id`
- `(student_id, name)`
- `(student_id, age)`

Because all combinations still uniquely identify rows.